

Praktikum jobsheet 11:

kode di class Mahasiswa23.java:

```
package Pertemuan11;

public class Mahasiswa23 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa23() {
    }

    public Mahasiswa23(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    void tampilInformasi() {
        System.out.println("NIM    : " + nim);
        System.out.println("Nama  : " + nama);
        System.out.println("Kelas : " + kelas);
        System.out.println("IPK   : " + ipk);
    }
}
```

Kode di class Node23.java:

```
package Pertemuan11;

public class Node23 {
    Mahasiswa23 data;
    Node23 next;

    public Node23(Mahasiswa23 data, Node23 next) {
        this.data = data;
        this.next = next;
    }
}
```

```
}
```

kode di class SingleLinkedList23.java:

```
package Pertemuan11;

public class SingleLinkedList23 {
    Node23 head;
    Node23 tail;

    boolean isEmpty() {
        return head == null;
    }

    void print() {
        if (isEmpty()) {
            System.out.println("Linked List Kosong");
            return;
        }

        Node23 tmp = head;
        while (tmp != null) {
            tmp.data.tampilInformasi();
            System.out.println();
            tmp = tmp.next;
        }
    }

    void addFirst(Mahasiswa23 input) {
        Node23 ndInput = new Node23(input, null);
        if (isEmpty()) {
            head = tail = ndInput;
        } else {
            ndInput.next = head;
            head = ndInput;
        }
    }

    void addLast(Mahasiswa23 input) {
        Node23 ndInput = new Node23(input, null);
        if (isEmpty()) {
            head = tail = ndInput;
        }
    }
}
```

```

        } else {
            tail.next = ndInput;
            tail = ndInput;
        }
    }

    void insertAfter(String keyNim, Mahasiswa23 input) {
        Node23 tmp = head;
        while (tmp != null) {
            if (tmp.data != null &&
tmp.data.nim.equalsIgnoreCase(keyNim)) {
                Node23 ndInput = new Node23(input, tmp.next);
                tmp.next = ndInput;
                if (tmp == tail) {
                    tail = ndInput;
                }
                return;
            }
            tmp = tmp.next;
        }
        System.out.println("Data key (NIM) tidak ditemukan: " +
keyNim);
    }

    void insertAt(int index, Mahasiswa23 input) {
        if (index < 0) {
            System.out.println("Index tidak valid.");
            return;
        }
        if (index == 0) {
            addFirst(input);
            return;
        }

        Node23 tmp = head;
        int i = 0;
        while (tmp != null && i < index - 1) {
            tmp = tmp.next;
            i++;
        }

        if (tmp == null) {
            System.out.println("Index melebihi panjang linked list.");

```

```

        return;
    }

    Node23 ndInput = new Node23(input, tmp.next);
    tmp.next = ndInput;
    if (ndInput.next == null) {
        tail = ndInput;
    }
}

Mahasiswa23 getData(int index) {
    if (index < 0 || isEmpty()) {
        return null;
    }
    Node23 tmp = head;
    int i = 0;
    while (tmp != null && i < index) {
        tmp = tmp.next;
        i++;
    }
    return tmp == null ? null : tmp.data;
}

int indexOf(String keyNim) {
    Node23 tmp = head;
    int i = 0;
    while (tmp != null) {
        if (tmp.data != null &&
tmp.data.nim.equalsIgnoreCase(keyNim)) {
            return i;
        }
        tmp = tmp.next;
        i++;
    }
    return -1;
}

void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List Kosong");
        return;
    }
    head = head.next;

```

```

        if (head == null) {
            tail = null;
        }
    }

    void removeLast() {
        if (isEmpty()) {
            System.out.println("Linked List Kosong");
            return;
        }
        if (head == tail) {
            head = tail = null;
            return;
        }

        Node23 tmp = head;
        while (tmp.next != tail) {
            tmp = tmp.next;
        }
        tmp.next = null;
        tail = tmp;
    }

    void remove(String keyNim) {
        if (isEmpty()) {
            System.out.println("Linked List Kosong");
            return;
        }

        if (head.data != null &&
head.data.nim.equalsIgnoreCase(keyNim)) {
            removeFirst();
            return;
        }

        Node23 tmp = head;
        while (tmp.next != null) {
            if (tmp.next.data != null &&
tmp.next.data.nim.equalsIgnoreCase(keyNim)) {
                tmp.next = tmp.next.next;
                if (tmp.next == null) {
                    tail = tmp;
                }
            }
        }
    }

```

```

        break;
    }
    tmp = tmp.next;
}
}

void removeAt(int index) {
    if (index < 0 || isEmpty()) {
        System.out.println("Index tidak valid / Linked List
kosong");
        return;
    }
    if (index == 0) {
        removeFirst();
        return;
    }

    Node23 tmp = head;
    int i = 0;
    while (tmp != null && i < index - 1) {
        tmp = tmp.next;
        i++;
    }
    if (tmp == null || tmp.next == null) {
        System.out.println("Index melebihi panjang linked list.");
        return;
    }
    if (tmp.next == tail) {
        tail = tmp;
    }
    tmp.next = tmp.next.next;
}
}

```

Kode di class java SLLmain.java:

```

package Pertemuan11;

import java.util.Scanner;

public class SLLMain23 {
    static String readLineNonEmpty(Scanner sc, String prompt) {

```

```

        String s;
        do {
            System.out.print(prompt);
            s = sc.nextLine().trim();
        } while (s.isEmpty());
        return s;
    }

    static double readDoubleFlexible(Scanner sc, String prompt) {
        System.out.print(prompt);
        String token = sc.nextLine().trim().replace(',', '.', '.');
        while (token.isEmpty()) {
            System.out.print(prompt);
            token = sc.nextLine().trim().replace(',', '.', '.');
        }
        return Double.parseDouble(token);
    }

    static Mahasiswa23 inputMahasiswa(Scanner sc) {
        String nim = readLineNonEmpty(sc, "NIM : ");
        String nama = readLineNonEmpty(sc, "Nama : ");
        String kelas = readLineNonEmpty(sc, "Kelas : ");
        double ipk = readDoubleFlexible(sc, "IPK : ");
        return new Mahasiswa23(nim, nama, kelas, ipk);
    }

    public static void main(String[] args) {
        SingleLinkedList23 singLL = new SingleLinkedList23();

        System.out.println("Data awal:");
        singLL.print();

        // Data awal sesuai instruksi (4 mahasiswa)
        Mahasiswa23 mhs1 = new Mahasiswa23("111", "Andi", "TI-1A",
3.45);
        Mahasiswa23 mhs2 = new Mahasiswa23("112", "Budi", "TI-1A",
3.70);
        Mahasiswa23 mhs3 = new Mahasiswa23("113", "Citra", "TI-1A",
3.20);
        Mahasiswa23 mhs4 = new Mahasiswa23("114", "Dina", "TI-1A",
3.90);

        System.out.println("\nAddFirst(mhs1)");
    }

```

```

singLL.addFirst(mhs1);
singLL.print();

System.out.println("\nAddLast(mhs2)");
singLL.addLast(mhs2);
singLL.print();

System.out.println("\nInsertAfter(key=111, mhs3)");
singLL.insertAfter("111", mhs3);
singLL.print();

System.out.println("\nInsertAt(index=2, mhs4)");
singLL.insertAt(2, mhs4);
singLL.print();

// Modifikasi pertanyaan 2.1.2 no 3: input dari keyboard
Scanner sc = new Scanner(System.in);
System.out.println("\n=== Input 1 data mahasiswa dari keyboard
(modifikasi pertanyaan) ===");
Mahasiswa23 mhsInput = inputMahasiswa(sc);
singLL.addLast(mhsInput);
System.out.println("\nSetelah addLast(data input):");
singLL.print();

// Percobaan 2: akses dan hapus
System.out.println("\n=== Percobaan 2: Akses & Hapus Data
===");

System.out.println("Data pada index 0:");
Mahasiswa23 d0 = singLL.getData(0);
if (d0 != null) {
    d0.tampilInformasi();
}

String cariNim = readLineNonEmpty(sc, "\nCari indexOf
berdasarkan NIM: ");
int idx = singLL.indexOf(cariNim);
System.out.println("Index NIM " + cariNim + " = " + idx);

System.out.println("\nRemoveFirst()");
singLL.removeFirst();
singLL.print();

System.out.println("\nRemoveLast()");

```

```

        singLL.removeLast();
        singLL.print();

        String hapusNim = readLineNonEmpty(sc, "\nRemove berdasarkan
NIM: ");
        singLL.remove(hapusNim);
        singLL.print();

        System.out.print("\nRemoveAt index: ");
        int hapusIdx = Integer.parseInt(sc.nextLine().trim());
        singLL.removeAt(hapusIdx);
        singLL.print();

        sc.close();
    }
}

```

Hasil:

addFirst dan addLast:

```

ava
Data awal:
Linked List Kosong

AddFirst(mhs1)
NIM   : 111
Nama  : Andi
Kelas : TI-1A
IPK   : 3.45

AddLast(mhs2)
NIM   : 111
Nama  : Andi
Kelas : TI-1A
IPK   : 3.45

NIM   : 112
Nama  : Budi
Kelas : TI-1A
IPK   : 3.7

```

```
InsertAfter(key=111, mhs3)
```

```
NIM : 111  
Nama : Andi  
Kelas : TI-1A  
IPK : 3.45
```

```
NIM : 113  
Nama : Citra  
Kelas : TI-1A  
IPK : 3.2
```

```
NIM : 112  
Nama : Budi  
Kelas : TI-1A  
IPK : 3.7
```

InsertAT:

```
InsertAt(index=2, mhs4)
```

```
NIM : 111  
Nama : Andi  
Kelas : TI-1A  
IPK : 3.45
```

```
NIM : 113  
Nama : Citra  
Kelas : TI-1A  
IPK : 3.2
```

```
NIM : 114  
Nama : Dina  
Kelas : TI-1A  
IPK : 3.9
```

```
NIM : 112  
Nama : Budi  
Kelas : TI-1A  
IPK : 3.7
```

Input 1 data mahasiswa:

```
=== Input 1 data mahasiswa dari keyboard (modifikasi pertanyaan) ===  
NIM   : 214524  
Nama  : eiojfe  
Kelas : efki  
IPK   : 4  
  
Setelah addLast(data input):  
NIM   : 111  
Nama  : Andi  
Kelas : TI-1A  
IPK   : 3.45  
  
NIM   : 113  
Nama  : Citra  
Kelas : TI-1A  
IPK   : 3.2  
  
NIM   : 114  
Nama  : Dina  
Kelas : TI-1A  
IPK   : 3.9
```

AddLast(input):

```
Setelah addLast(data input):  
NIM   : 111  
Nama  : Andi  
Kelas : TI-1A  
IPK   : 3.45  
  
NIM   : 113  
Nama  : Citra  
Kelas : TI-1A  
IPK   : 3.2  
  
NIM   : 114  
Nama  : Dina  
Kelas : TI-1A  
IPK   : 3.9  
  
NIM   : 112  
Nama  : Budi  
Kelas : TI-1A  
IPK   : 3.7  
  
NIM   : 214524  
Nama  : eiojfe  
Kelas : efki  
IPK   : 4.0
```

SearchData di index:

```
=== Percobaan 2: Akses & Hapus Data ===  
Data pada index 0:  
NIM   : 111  
Nama  : Andi  
Kelas : TI-1A  
IPK   : 3.45
```

RemoveFirst()

```
RemoveFirst()
```

```
NIM   : 113  
Nama  : Citra  
Kelas : TI-1A  
IPK   : 3.2
```

```
NIM   : 114  
Nama  : Dina  
Kelas : TI-1A  
IPK   : 3.9
```

```
NIM   : 112  
Nama  : Budi  
Kelas : TI-1A  
IPK   : 3.7
```

```
NIM   : 214524  
Nama  : eiojfe  
Kelas : efki  
IPK   : 4.0
```

removeLast()

```
RemoveLast()
```

```
NIM   : 113  
Nama  : Citra  
Kelas : TI-1A  
IPK   : 3.2
```

```
NIM   : 114  
Nama  : Dina  
Kelas : TI-1A  
IPK   : 3.9
```

```
NIM   : 112  
Nama  : Budi  
Kelas : TI-1A  
IPK   : 3.7
```

removeNim()

```
Remove berdasarkan NIM: 112
NIM   : 113
Nama  : Citra
Kelas : TI-1A
IPK   : 3.2

NIM   : 114
Nama  : Dina
Kelas : TI-1A
IPK   : 3.9
```

remove at index:

```
RemoveAt index: 3
Index melebihi panjang linked list.
NIM   : 113
Nama  : Citra
Kelas : TI-1A
IPK   : 3.2

NIM   : 114
Nama  : Dina
Kelas : TI-1A
IPK   : 3.9
```

2.1.2 Pertanyaan

Percobaan 1 – Pembuatan Single Linked List

1) Mengapa baris pertama menampilkan “Linked List Kosong”?

Karena pada awal program, linked list masih belum berisi node (`head == null`). Saat `print()` dipanggil, `isEmpty()` bernilai `true`, sehingga program menampilkan teks “Linked List Kosong”.

2) Jelaskan kegunaan variabel `temp/tmp` secara umum pada setiap method!

`temp/tmp` digunakan sebagai pointer sementara untuk:

- melakukan traversal (menelusuri node dari head sampai null),
 - mencari node tertentu (misalnya node dengan nim tertentu),
 - berhenti di node sebelum target (misalnya saat insert/remove),
- tanpa mengubah referensi head/tail.

3) Modifikasi agar data dapat ditambahkan dari keyboard

Modifikasi dilakukan dengan menambahkan input Scanner pada class main, lalu membaca data mahasiswa dan memanggil `addLast()` (atau method tambah lain).

Kode yang dimodifikasi (jawaban diminta menyebutkan perubahan):

- File: `SLLMain23.java`

- Ditambahkan Scanner sc + helper inputMahasiswa() untuk input dari keyboard:

```
import java.util.Scanner;
```

```
static Mahasiswa23 inputMahasiswa(Scanner sc) {  
    String nim = readLineNonEmpty(sc, "NIM : ");  
    String nama = readLineNonEmpty(sc, "Nama : ");  
    String kelas = readLineNonEmpty(sc, "Kelas : ");  
    double ipk = readDoubleFlexible(sc, "IPK : ");  
    return new Mahasiswa23(nim, nama, kelas, ipk);  
}
```

- Ditambahkan blok:

```
System.out.println("\n=== Input 1 data mahasiswa dari keyboard  
(modifikasi pertanyaan) ===");  
    Mahasiswa23 mhsInput = inputMahasiswa(sc);  
    singLL.addLast(mhsInput);
```

Percobaan 2 – Modifikasi Elemen pada Single Linked List

2.2.3 Pertanyaan

1) Mengapa digunakan keyword break pada fungsi remove?

break dipakai untuk menghentikan traversal setelah node yang dicari berhasil dihapus.

Tanpa break, perulangan akan terus berjalan dan berpotensi:

- menghapus lebih dari satu node (jika ada NIM yang sama),
- atau menyebabkan logika traversal menjadi tidak konsisten setelah pointer next diubah.

2) Jelaskan kegunaan kode berikut pada method remove

Potongan kode (sesuai implementasi di remove()):

```
java  
tmp.next = tmp.next.next;  
if (tmp.next == null) {  
    tail = tmp;  
}  
break;
```

Penjelasan:

- tmp.next = tmp.next.next; → melewati (skip) node yang dihapus, sehingga node tersebut tidak lagi terhubung di linked list.

- if (tmp.next == null) tail = tmp; → jika node yang dihapus adalah tail lama, maka tail harus dipindah ke node sebelumnya (tmp).

- break; → berhenti karena penghapusan sudah selesai.

Kode yang dimodifikasi:

- Percobaan 2 membutuhkan penambahan method akses/hapus. Implementasi ada di:

- File:SingleLinkedList23.java

- ditambahkan:

getData(int)

```
Mahasiswa23 getData(int index) {  
    if (index < 0 || isEmpty()) {  
        return null;  
    }  
    Node23 tmp = head;  
    int i = 0;  
    while (tmp != null && i < index) {  
        tmp = tmp.next;  
        i++;  
    }  
    return tmp == null ? null : tmp.data;  
}
```

indexOf(String)

```
int indexOf(String keyNim) {  
    Node23 tmp = head;  
    int i = 0;  
    while (tmp != null) {  
        if (tmp.data != null &&  
tmp.data.nim.equalsIgnoreCase(keyNim)) {  
            return i;  
        }  
        tmp = tmp.next;  
        i++;  
    }  
    return -1;  
}
```

removeFirst()

```
void removeFirst() {  
    if (isEmpty()) {  
        System.out.println("Linked List Kosong");  
        return;  
    }  
    head = head.next;  
    if (head == null) {  
        tail = null;  
    }  
}
```

```
}  
}
```

removeLast()

```
void removeLast() {  
    if (isEmpty()) {  
        System.out.println("Linked List Kosong");  
        return;  
    }  
    if (head == tail) {  
        head = tail = null;  
        return;  
    }  
  
    Node23 tmp = head;  
    while (tmp.next != tail) {  
        tmp = tmp.next;  
    }  
    tmp.next = null;  
    tail = tmp;  
}
```

remove(String)

```
void remove(String keyNim) {  
    if (isEmpty()) {  
        System.out.println("Linked List Kosong");  
        return;  
    }  
  
    if (head.data != null &&  
head.data.nim.equalsIgnoreCase(keyNim)) {  
        removeFirst();  
        return;  
    }  
  
    Node23 tmp = head;  
    while (tmp.next != null) {  
        if (tmp.next.data != null &&  
tmp.next.data.nim.equalsIgnoreCase(keyNim)) {  
            tmp.next = tmp.next.next;  
            if (tmp.next == null) {
```

```

        tail = tmp;
    }
    break;
}
tmp = tmp.next;
}
}

```

removeAt(int)

```

void removeAt(int index) {
    if (index < 0 || isEmpty()) {
        System.out.println("Index tidak valid / Linked List
kosong");
        return;
    }
    if (index == 0) {
        removeFirst();
        return;
    }

    Node23 tmp = head;
    int i = 0;
    while (tmp != null && i < index - 1) {
        tmp = tmp.next;
        i++;
    }
    if (tmp == null || tmp.next == null) {
        System.out.println("Index melebihi panjang linked list.");
        return;
    }
    if (tmp.next == tail) {
        tail = tmp;
    }
    tmp.next = tmp.next.next;
}
}

```

- File: SLLMain23.java

- ditambahkan pemanggilan method-method di atas untuk uji akses & hapus.

```

System.out.println("\nRemoveFirst()");
singLL.removeFirst();

```

```

        singLL.print();

        System.out.println("\nRemoveLast()");
        singLL.removeLast();
        singLL.print();

        String hapusNim = readLineNonEmpty(sc, "\nRemove berdasarkan
NIM: ");
        singLL.remove(hapusNim);
        singLL.print();

        System.out.print("\nRemoveAt index: ");
        int hapusIdx = Integer.parseInt(sc.nextLine().trim());
        singLL.removeAt(hapusIdx);
        singLL.print();

```

Tugas – Antrian Layanan Unit Kemahasiswaan (Queue berbasis Linked List)

kode di MahasiswaAntri23.java:

```

package Pertemuan11.TugasAntrianKemahasiswaan;

public class MahasiswaAntri23 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public MahasiswaAntri23(String nim, String nama, String prodi,
String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    void tampilkanData() {
        System.out.println("NIM    : " + nim);
        System.out.println("Nama  : " + nama);
        System.out.println("Prodi : " + prodi);
        System.out.println("Kelas : " + kelas);
    }
}

```

```
}
```

Kode di NodeAntri23.java:

```
package Pertemuan11.TugasAntrianKemahasiswaan;

public class NodeAntri23 {
    MahasiswaAntri23 data;
    NodeAntri23 next;

    public NodeAntri23(MahasiswaAntri23 data) {
        this.data = data;
        this.next = null;
    }
}
```

Kode di QueueLinkedList23.java:

```
package Pertemuan11.TugasAntrianKemahasiswaan;

public class QueueLinkedList23 {
    NodeAntri23 front;
    NodeAntri23 rear;
    int size;
    int max;

    public QueueLinkedList23(int max) {
        this.max = max;
        this.front = null;
        this.rear = null;
        this.size = 0;
    }

    boolean isEmpty() {
        return size == 0;
    }

    boolean isFull() {
        return size == max;
    }
}
```

```

void clear() {
    front = rear = null;
    size = 0;
    System.out.println("Antrian berhasil dikosongkan.");
}

void enqueue(MahasiswaAntri23 mhs) {
    if (isFull()) {
        System.out.println("Antrian penuh! Tidak bisa menambah.");
        return;
    }

    NodeAntri23 newNode = new NodeAntri23(mhs);
    if (isEmpty()) {
        front = rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
    size++;
    System.out.println("Mahasiswa berhasil ditambahkan ke antrian.");
}

MahasiswaAntri23 dequeue() {
    if (isEmpty()) {
        System.out.println("Antrian kosong! Tidak ada yang dipanggil.");
        return null;
    }

    MahasiswaAntri23 data = front.data;
    front = front.next;
    size--;
    if (front == null) {
        rear = null;
    }
    return data;
}

MahasiswaAntri23 peekFront() {
    return isEmpty() ? null : front.data;
}

```

```

    }

    MahasiswaAntri23 peekRear() {
        return isEmpty() ? null : rear.data;
    }

    int getJumlahAntrian() {
        return size;
    }
}

```

Kode di AntriankemahasiswaanMain23.java:

```

package Pertemuan11.TugasAntrianKemahasiswaan;

import java.util.Scanner;

public class AntrianKemahasiswaanMain23 {
    static int readInt(Scanner sc, String prompt) {
        System.out.print(prompt);
        while (!sc.hasNextInt()) {
            sc.next();
            System.out.print(prompt);
        }
        int val = sc.nextInt();
        sc.nextLine();
        return val;
    }

    static String readLineNonEmpty(Scanner sc, String prompt) {
        String s;
        do {
            System.out.print(prompt);
            s = sc.nextLine().trim();
        } while (s.isEmpty());
        return s;
    }

    static void menu() {
        System.out.println("\n=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===");
        System.out.println("1. Daftar & Tambah Antrian");
    }
}

```

```

        System.out.println("2. Panggil Antrian (Dequeue)");
        System.out.println("3. Cek Antrian Kosong");
        System.out.println("4. Cek Antrian Penuh");
        System.out.println("5. Kosongkan Antrian");
        System.out.println("6. Lihat Antrian Terdepan");
        System.out.println("7. Lihat Antrian Paling Akhir");
        System.out.println("8. Cetak Jumlah Mahasiswa yang Masih
Mengantrre");
        System.out.println("0. Keluar");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        QueueLinkedList23 antrian = new QueueLinkedList23(10);

        int pilih;
        do {
            menu();
            pilih = readInt(sc, "Pilih: ");
            switch (pilih) {
                case 1: {
                    String nim = readLineNonEmpty(sc, "NIM : ");
                    String nama = readLineNonEmpty(sc, "Nama : ");
                    String prodi = readLineNonEmpty(sc, "Prodi : ");
                    String kelas = readLineNonEmpty(sc, "Kelas : ");
                    antrian.enqueue(new MahasiswaAntri23(nim, nama,
prodi, kelas));
                    break;
                }
                case 2: {
                    MahasiswaAntri23 dipanggil = antrian.dequeue();
                    if (dipanggil != null) {
                        System.out.println("\nMahasiswa dipanggil:");
                        dipanggil.tampilkanData();
                    }
                    break;
                }
                case 3:
                    System.out.println("Antrian kosong: " +
antrian.isEmpty());
                    break;
                case 4:

```

```

        System.out.println("Antrian penuh: " +
antrian.isFull());
        break;
    case 5:
        antrian.clear();
        break;
    case 6: {
        MahasiswaAntri23 depan = antrian.peekFront();
        if (depan == null) {
            System.out.println("Antrian kosong.");
        } else {
            System.out.println("\nAntrian terdepan:");
            depan.tampilkanData();
        }
        break;
    }
    case 7: {
        MahasiswaAntri23 belakang = antrian.peekRear();
        if (belakang == null) {
            System.out.println("Antrian kosong.");
        } else {
            System.out.println("\nAntrian paling akhir:");
            belakang.tampilkanData();
        }
        break;
    }
    case 8:
        System.out.println("Jumlah mahasiswa yang masih
mengantrre: " + antrian.getJumlahAntrian());
        break;
    case 0:
        System.out.println("Keluar.");
        break;
    default:
        System.out.println("Menu tidak tersedia.");
    }
} while (pilih != 0);

sc.close();
}
}

```

Hasil:

Daftar dan tambah antrian

```
asiswaan> java AntrianKemahasiswaanMain23.java

=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 1
NIM   : 111
Nama  : rozi
Prodi : ti
Kelas : 1g
Mahasiswa berhasil ditambahkan ke antrian.
```

Panggilan antrian (dequeue):

```
Mahasiswa berhasil ditambahkan ke antrian.

=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 2

Mahasiswa dipanggil:
NIM   : 111
Nama  : rozi
Prodi : ti
Kelas : 1g
```

Cek antrian kosong:

```
=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 3
Antrian kosong: true
```

cek antrian penuh

```
=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 4
Antrian penuh: false
```

kosongkan antrian

```
=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 5
Antrian berhasil dikosongkan.
```

lihat antrian terdepan

```
=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 6
Antrian kosong.
```

lihat antrian paling akhir

```
=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 7
Antrian kosong.
```

cetak jumlah mahasiswa

```
Antrian kosong.

=== Antrian Layanan Unit Kemahasiswaan (Queue Linked List) ===
1. Daftar & Tambah Antrian
2. Panggil Antrian (Dequeue)
3. Cek Antrian Kosong
4. Cek Antrian Penuh
5. Kosongkan Antrian
6. Lihat Antrian Terdepan
7. Lihat Antrian Paling Akhir
8. Cetak Jumlah Mahasiswa yang Masih Mengantre
0. Keluar
Pilih: 8
Jumlah mahasiswa yang masih mengantre: 0
```